

C++ 프로그래밍

(프렌드와 연산자 중복)

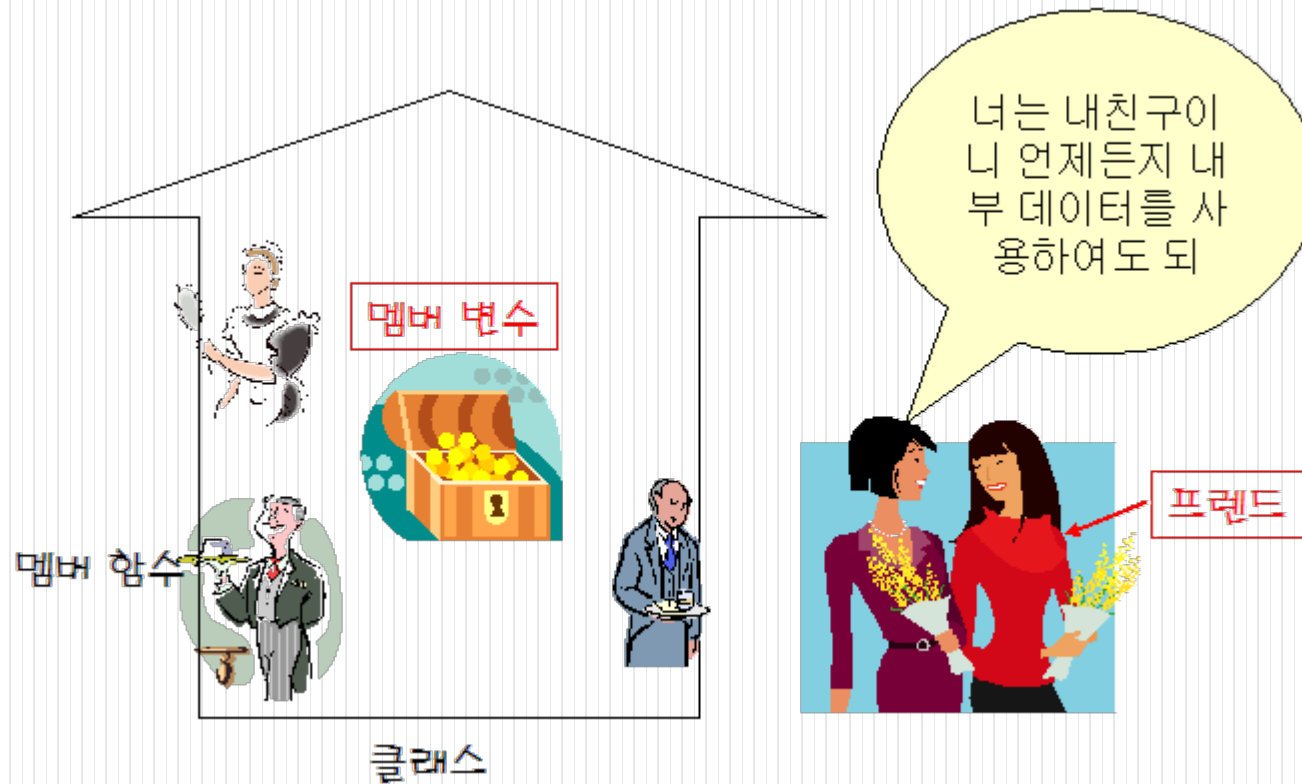
9주차

목차

- 프렌드 함수
- 연산자 중복
- 타입 변환

프렌드 함수

- 프렌드 함수란?
 - 클래스의 내부 데이터에 접근할 수 있는 특수한 함수



프렌드 함수 (계속)

- 프렌드 함수 선언 방법
 - 프렌드 함수의 원형은 비록 클래스 안에 포함.
 - 하지만 멤버 함수는 아니다.
 - 프렌드 함수의 본체는 외부에서 따로 정의
 - 프렌드 함수는 클래스 내부의 모든 멤버 변수를 사용 가능

```
class MyClass
{
    friend void sub();
    ....
};
```



프렌드 함수

프렌드 함수 (계속)

- 예제

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Company {
private:
    int sales, profit;
```

```
// sub()는 Company의 전용부분에 접근할 수 있다.
friend void sub(Company& c);
```

```
public:
    Company(): sales(0), profit(0)
    {
    }
};
void sub(Company& c)
{
    cout << c.profit << endl;
}
```

```
int main()
{
    Company c1;
    sub(c1);
    return 0;
}
```

```
0
```

프렌드 함수 (계속)

- 프렌드 클래스
 - 클래스도 프렌드로 선언할 수 있다.
 - Manager의 멤버들은 Employee의 전용 멤버를 직접 참조할 수 있다.

```
class Employee {  
    int salary;  
    // Manager는 Employee의 전용 부분에 접근할 수 있다.  
    friend class Manager;  
    // ...  
};
```

프렌드 클래스

프렌드 함수 (계속)

- 프렌드 함수의 용도
 - 두개의 객체를 비교할 때 많이 사용

① 일반 멤버 함수 사용

```
if( obj1.equals(obj2) )  
{  
    ...  
}
```

② 프렌드 함수 사용

```
if( equals(obj1, obj2) )  
{  
    ...  
}
```

이해하기가 쉽다

프렌드 함수 (계속)

- 예제

```
#include <iostream>
using namespace std;
```

```
class Date
```

```
{
```

```
private:
```

```
    int year, month, day;
```

```
public:
```

```
    Date(int y, int m, int d)
```

```
    {
```

```
        year = y;
        month = m;
        day = d;
```

```
    }
```

```
}
```

```
// 프렌드 함수
```

```
bool equals(Date d1, Date d2)
```

```
{
```

```
    return d1.year == d2.year && d1.month == d2.month && d1.day == d2.day;
```

```
}
```

```
int main()
```

```
{
```

```
    Date d1(1960, 5, 23), d2(2002, 7, 23);
    cout << equal_f(d1, d2) << endl;
```

```
}
```

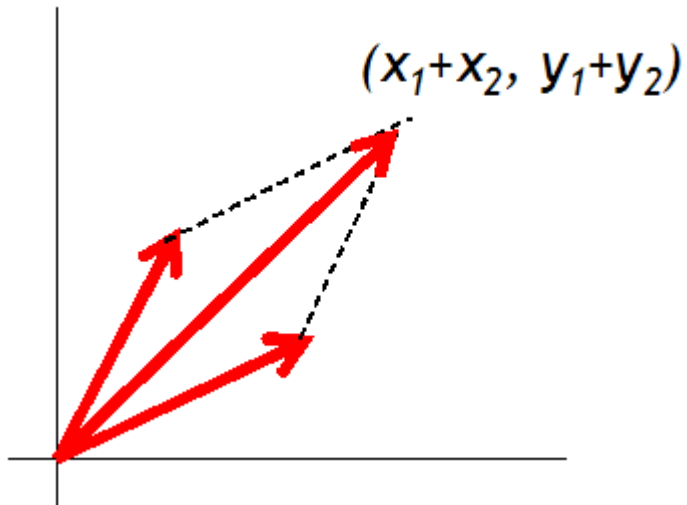
멤버 변수 접근
가능

연산자 중복

- 연산자 중복
 - 일반적으로는 연산자 기호를 사용하는 편이 함수를 사용하는 것보다 이해하기가 쉽다.
- 두가지 문자 중에서 어떤 것이 더 하기 쉬운가?
 - `sum = x+y+z;`
 - `sum = add(x, add(y,z));`

연산자 중복 (계속)

- 벡터간의 연산을 연산자로 표기
 - vector v_1, v_2, v_3 ;
 - $v_3 = v_1 + v_2$;



연산자 중복 (계속)

- 연산자 중복(operator overloading)
 - 여러 가지 연산자들을 클래스 객체에 대해서도 적용하는 것
 - C++에서 연산자는 함수로 정의

```
반환형 operator연산자(매개 변수 목록)
{
    ....// 연산 수행
}
```

(예) Vector operator+(const Vector&, const Vector&);

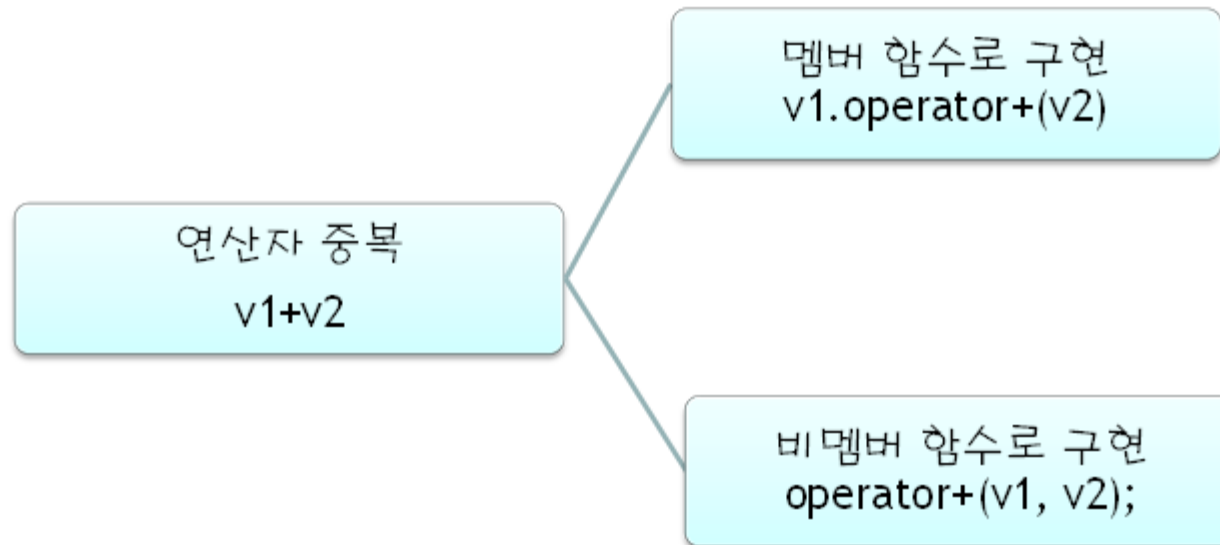
연산자 중복 (계속)

- 연산자 중복 함수 이름
 - 중복 함수 이름은 operator에 연산자를 붙이고 함수 기호

연산자	중복 함수 이름
+	operator+()
-	operator-()
*	operator*()
/	operator/()

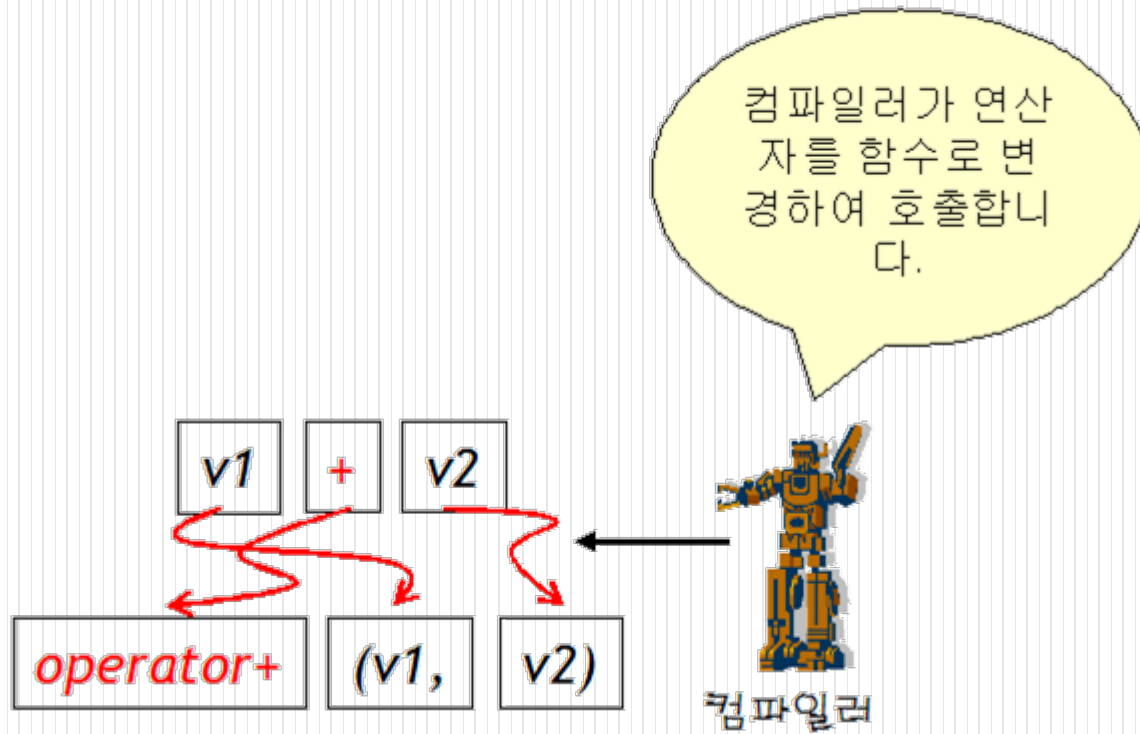
연산자 중복 (계속)

- 연산자 중복 구현의 방법



연산자 중복 (계속)

- 연산자 중복 구현의 방법 (계속)



연산자 중복 (계속)

- 예제

```
#include <iostream>
using namespace std;

class Vector
{
private:
    double x, y;
public:
    Vector(double x, double y){
        this->x = x;
        this->y = y;
    }
    friend Vector operator+(const Vector& v1, const Vector& v2);
    void display()
    {
        cout << "(" << x << ", " << y << ")" << endl;
    }
};
```

```
Vector operator+(const Vector& v1, const Vector& v2)
{
    Vector v(0.0, 0.0);
    v.x = v1.x + v2.x;
    v.y = v1.y + v2.y;
    return v;
}
```

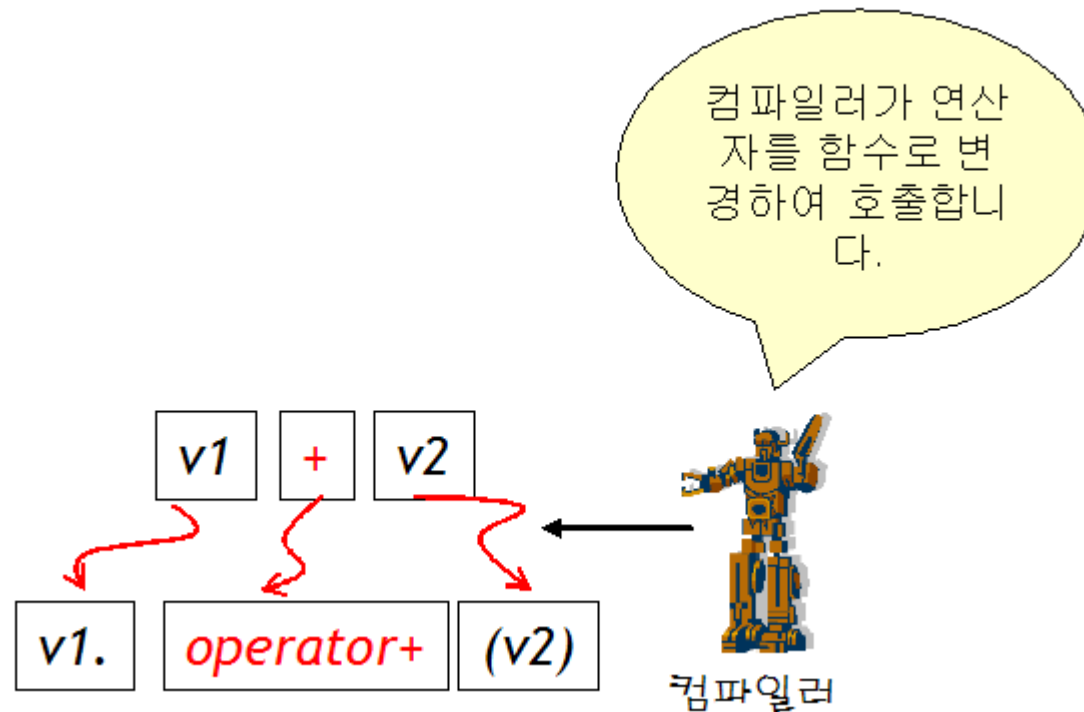
```
int main()
{
    Vector v1(1, 2), v2(3, 4);
    Vector v3 = v1 + v2;
    v3.display();

    return 0;
}
```

(4, 6)

연산자 중복 (계속)

- 연산자 중복 구현의 방법 (계속)



연산자 중복 (계속)

- 예제

```
#include <iostream>
using namespace std;
```

```
class Vector
```

```
{
```

```
private:
```

```
    double x, y;
```

```
public:
```

```
    Vector(double x, double y){
        this->x = x;
```

```
        this->y = y;
```

```
    }
```

```
    Vector operator+(Vector& v2)
```

```
    {
```

```
        Vector v(0.0, 0.0);
```

```
        v.x = this->x + v2.x;
```

```
        v.y = this->y + v2.y;
```

```
        return v;
```

```
    }
```

```
void display()
```

```
{
```

```
    cout << "(" << x << ", " << y << ")" << endl;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    Vector v1(1.0, 2.0), v2(3.0, 4.0);
```

```
    Vector v3 = v1 + v2;
```

```
    v3.display();
```

```
    return 0;
```

```
}
```

(4, 6)

연산자 중복 (계속)

- 멤버 함수로만 구현가능한 연산자
 - 연산자는 항상 멤버 함수 형태로만 중복 정의가 가능

연산자	설명
=	대입 연산자
()	함수 호출 연산자
[]	배열 원소 참조 연산자
->	멤버 참조 연산자

연산자 중복 (계속)

- 중복이 불가능한 연산자
 - 연산자는 중복 정의가 불가능

연산자	설명
::	범위 지정 연산자
.	멤버 선택 연산자
.*	멤버 포인터 연산자
?:	조건 연산자

연산자 중복 (계속)

- 곱셈 연산자 중복
 - 곱셈 연산자를 중복하여 정의
 - 교환 법칙이 성립하여야 함.

Vector operator*(Vector& v, double alpha); // $v * 2.0$ 형태 처리

Vector operator*(double alpha, Vector& v); // $2.0 * v$ 형태 처리

연산자 중복 (계속)

● 예제

```
#include <iostream>
using namespace std;

class Vector
{
    friend Vector operator*(Vector& v, double alpha);
    friend Vector operator*(double alpha, Vector& v);
private:
    double x, y;
public:
    Vector(double xvalue=0.0, double yvalue=0.0) : x(xvalue), y(yvalue){ }
    void display(){
        cout << "(" << x << ", " << y << ")" << endl;
    }
};
```

```
Vector operator*(Vector& v, double alpha)
{
    return Vector(alpha*v.x, alpha*v.y);
}
```

* 연산자 함수 정의

```
Vector operator*(double alpha, Vector& v)
{
    return Vector(alpha*v.x, alpha*v.y);
}
```

* 연산자 함수 정의

```
int main()
{
    Vector v(1.0, 1.0);
    Vector w = v * 2.0;
    Vector z = 2.0 * v;
    w.display();
    z.display();
    return 0;
}
```

연산자 중복 (계속)

- "==" 연산자 중복
 - 두개의 객체가 동일한 데이터를 가지고 있는지를 체크하는데 사용

연산자	중복 함수 이름
==	operator==()
!=	operator!=()

연산자 중복 (계속)

● 예제

```
#include <iostream>
using namespace std;

class Vector
{
private:
    double x, y;
public:
    Vector(double xvalue=0.0, double yvalue=0.0) : x(xvalue), y(yvalue){ }
    void display(){
        cout << "(" << x << ", " << y << ")" << endl;
    }

    friend bool operator==(const Vector& v1, const Vector& v2);
    friend bool operator!=(const Vector& v1, const Vector& v2);
};
```

```
//
bool operator==(const Vector &v1, const Vector &v2)
{
    return v1.x == v2.x && v2.y == v2.y;
}
bool operator!=(const Vector &v1, const Vector &v2)
{
    return !(v1 == v2); // 중복된 == 연산자를 이용
}
```

==와 !=연산자 함수를
전역 함수로 구현

```
int main()
{
    Vector v1(1, 2), v2(1, 2);

    cout.setf(cout.boolalpha);
    cout << (v1 == v2) << endl;
    cout << (v1 != v2) << endl;
    return 0;
}
```

연산자 중복 (계속)

- "<<" 연산자의 중복

```
Vector v(2, 3);
```

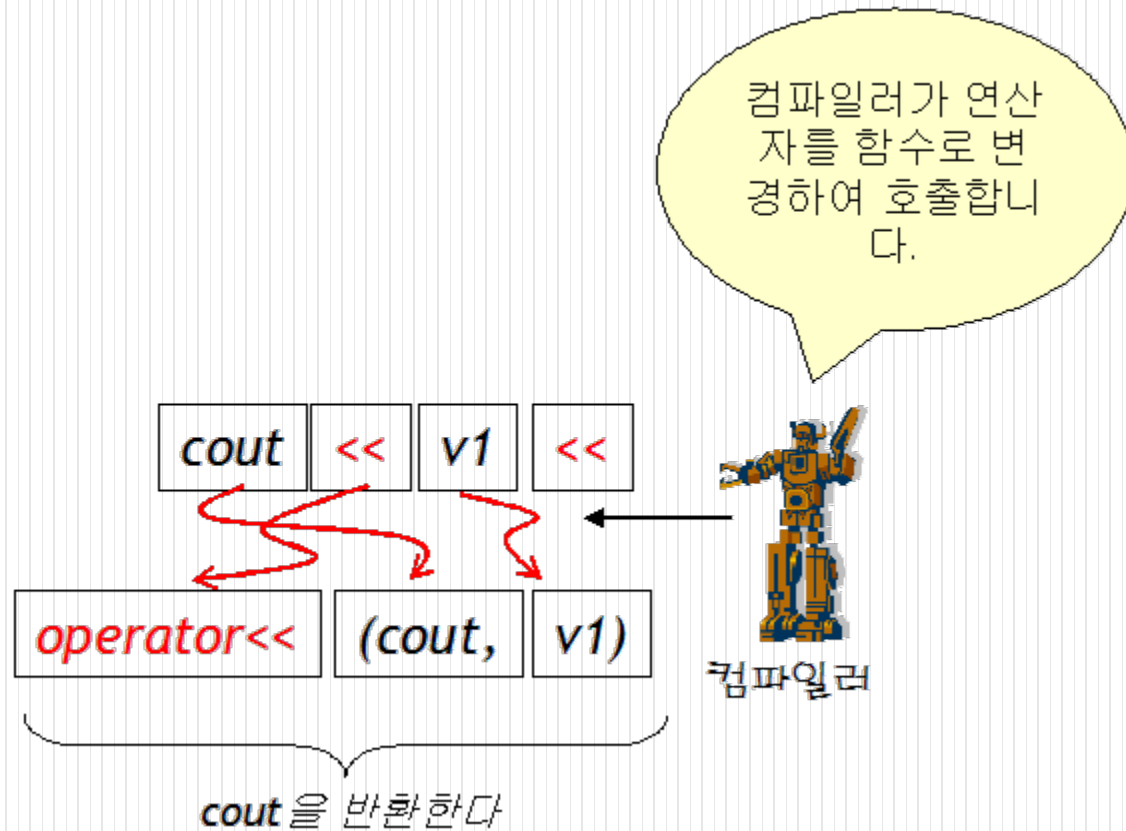
```
cout << v;
```

// 화면에 (2, 3)이 출력된다.



연산자 중복 (계속)

- "<<" 연산자의 중복 (계속)
 - 연산을 수행한 후에 다시 스트림 객체를 반환하여야 함.



연산자 중복 (계속)

- 예제

```
#include <iostream>
using namespace std;
class Vector
{
    friend ostream& operator<<(ostream& os, const Vector& v);
private:
    double x, y;
public:
    Vector(double xvalue=0.0, double yvalue=0.0) : x(xvalue), y(yvalue){ }
    void display(){
        cout << "(" << x << ", " << y << ")" << endl;
    }
};
```

```
ostream& operator<<(ostream& os, const Vector& v)
{
    os << "(" << v.x << ", " << v.y << ")" << endl;
    return os;
}
```

<< 연산자 함수 정의

```
int main()
{
    Vector v1(1.0, 2.0), v2(3.0, 4.0), v3;
    cout << v1 << v2 << v3;
    return 0;
}
```

실행결과

(1,2)

(3,4)

(0,0)

연산자 중복 (계속)

- ">>" 연산자의 중복
 - 입력 연산자 >>의 중복
 - 오류 처리를 하는 것이 좋음.

```
istream& operator>>(istream& in, Vector& v)
{
    in >> v.x >> v.y;
    if(!in)
        v = Vector(0, 0);
    return in;
}
```

입력 오류 처리

연산자 중복 (계속)

- "=" 연산자의 중복

```
class Vector
{
    ...
    Vector& operator=(const Vector& v2)
    {
        this->x = v2.x;
        this->y = v2.y;
        return *this;
    }
    ...
};
```

주의: 반드시 현재 객체의 레퍼런스를 반환

```
Vector v1(2.0, 3.0);
v3 = v2 = v1; // 가능!
```

연산자 중복 (계속)

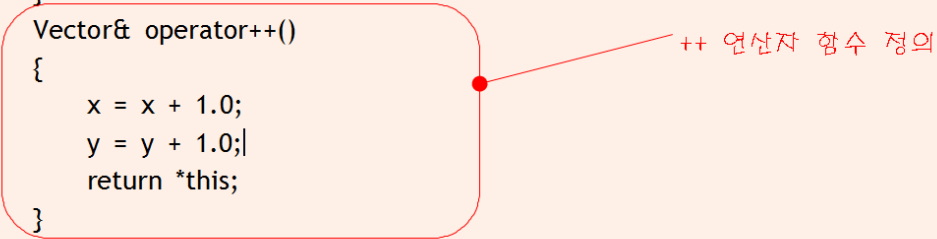
- 증가/감소 연산자의 중복
 - ++와 --연산자의 중복

연산자	중복 함수 이름
++v	<u>v.operator++()</u>
--v	<u>v.operator--()</u>

연산자 중복 (계속)

• 예제

```
#include <iostream>
using namespace std;
class Vector
{
private:
    double x, y;
public:
    Vector(double xvalue=0.0, double yvalue=0.0) : x(xvalue), y(yvalue){ }
    void display(){
        cout << "(" << x << ", " << y << ")" << endl;
    }
    Vector& operator++()
    {
        x = x + 1.0;
        y = y + 1.0;
        return *this;
    }
};
```



++ 연산자 함수 정의

```
int main()
{
    Vector v;
    ++v;
    v.display();
    ++(++v);
    v.display();

    return 0;
}
```

실행결과

```
(1, 1)
(3, 3)
```

연산자 중복 (계속)

- 전위/후위의 문제
 - 전위와 후위 연산자를 구별하기 위하여 ++가 피연산자 뒤에 오는 경우에는 int형 매개 변수를 추가 한다.

연산자	중복 함수 이름
++V	<u>v.operator++()</u>
V++	<u>v.operator++(int)</u>

```
Vector& operator++()  
{  
    x++;  
    y++;  
    return *this;  
}  
  
const Vector operator++(int)  
{  
    Vector saveObj = *this;  
    x++;  
    y++;  
    return saveObj;  
}  
};
```

++v 형태의 증가 연산자
함수 정의

v++ 형태의 증가 연산자
함수 정의

연산자 중복 (계속)

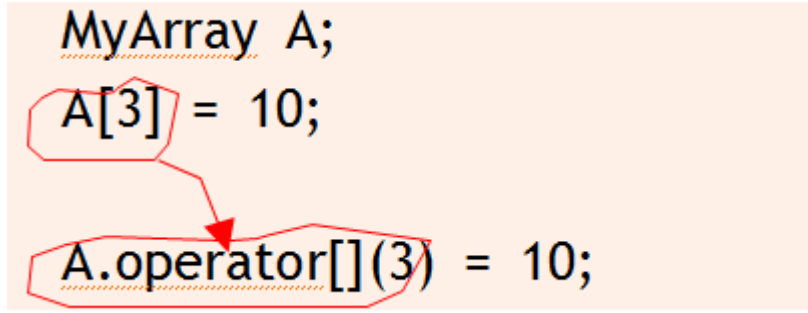
- [] 연산자 중복
 - 인덱스 연산자의 중복

연산자	중복 함수 이름
v[]	<u>v.operator[]()</u>

```
MyArray A;
```

```
A[3] = 10;
```

```
A.operator[](3) = 10;
```



연산자 중복 (계속)

● 예제

```
#include <iostream>
#include <assert.h>
using namespace std;

// 항상된 배열을 나타낸다.
class MyArray {
    friend ostream& operator<<(ostream &, const MyArray &);    // 출력 연산자 <<
private:
    int *data;          // 배열의 데이터
    int size;           // 배열의 크기

public:
    MyArray(int size = 10);    // 디폴트 생성자
    ~MyArray();               // 소멸자

    int getSize() const;      // 배열의 크기를 반환
    MyArray& operator=(const MyArray &a);    // = 연산자 중복 정의
    int& operator[](int i);    // [] 연산자 중복: 설정자
};
```

```
MyArray::MyArray(int s) {
    size = (s > 0 ? s : 10);    // 디폴트 크기를 10으로 한다.
    data = new int[size];       // 동적 메모리 할당

    for (int i = 0; i < size; i++)
        data[i] = 0;           // 요소들의 초기화
}

MyArray::~MyArray() {
    delete [] data;              // 동적 메모리 반납
    data = NULL;
}
```

연산자 중복 (계속)

• 예제 (계속)

```
MyArray& MyArray::operator=(const MyArray& a) {  
    if (&a != this) {  
        delete [] data;           // 자기 자신인지를 체크 의  
        size = a.size;           // 동적 메모리 반납  
        data = new int[size];     // 새로운 크기를 설정  
                                   // 새로운 동적 메모리 할당  
  
        for (int i = 0; i < size; i++)  
            data[i] = a.data[i];  // 데이터 복사  
    }  
    return *this;                // a = b = c와 같은 경우를 대비  
}
```

```
int MyArray::getSize() const  
{  
    return size;  
}  
  
int& MyArray::operator[](int index) {  
    assert(0 <= index && index < size); // 인덱스가 범위에 있지 않으면 중지  
    return data[index];  
}  
  
// 프렌드 함수 정의  
ostream& operator<<(ostream &output, const MyArray &a) {  
    int i;  
    for (i = 0; i < a.size; i++) {  
        output << a.data[i] << ' ';  
    }  
    output << endl;  
    return output;                // cout << a1 << a2 << a3와 같은 경우 대비  
}
```

연산자 중복 (계속)

- 예제 (계속)

```
int main()
{
    MyArray a1(10);

    a1[0] = 1;
    a1[1] = 2;
    a1[2] = 3;
    a1[3] = 4;
    cout << a1 ;

    return 0;
}
```

실행결과

1 2 3 4 0 0 0 0 0 0

연산자 중복 (계속)

- 포인터 연산자의 중복
 - 간접 참조 연산자 "*"와 멤버 연산자 "->"의 중복 정의

연산자	중복 함수 이름
*	operator*()
->	operator->()

연산자 중복 (계속)

- 예제

```
#include <iostream>
using namespace std;

class Pointer {
    int *pi;
public:
    Pointer(int *p): pi(p)
    {
    }
    ~Pointer()
    {
        delete pi;
    }
};
```

```
int* operator->() const
{
    return pi;
}
```

-> 연산자 중복 정의

```
int& operator*() const
{
    return *pi;
}
```

* 연산자 중복 정의

```
};
```

```
int main()
{
    Pointer p(new int);
    *p = 100;
    cout << *p << endl;
    return 0;
}
```

실행결과

100

계속하려면 아무 키나 누르십시오 . . .

연산자 중복 (계속)

- 스마트 포인터
 - 포인터 연산 정의를 이용하여서 만들어진 향상된 포인터를 스마트 포인터(smart pointer)라고 한다.
 - 주로 동적 할당된 공간을 반납할 때 사용된다.

```
class Pointer {  
    Car *pc;  
public:  
    Pointer(Car *p): pc(p){ }  
    ~Pointer(){ delete pc; }  
    Car* operator->() const { return pc; }  
    Car& operator*() const { return *pc; }  
};
```

스마트 포인터

```
int main()  
{  
    Pointer p(new Car(0,1,"red"));  
    p->speed = 100;  
    cout << *p;  
    (*p).speed = 200;  
    cout << *p;  
    p->setSpeed(300);  
    cout << *p;  
    return 0;  
}
```

연산자 중복 (계속)

- 함수 호출 연산자()의 중복
 - 함수 호출때 사용하는 ()도 중복이 가능하다.

연산자	중복 함수 이름
<u>f(...)</u>	<u>f.operator()(...)</u>

연산자 중복 (계속)

- 예제

```
#include <iostream>
#include <string>
using namespace std;

class Negate {
public:
    int operator()(int value) {
        return -value;
    }
};

int main()
{
    Negate neg;
    cout << neg(100) << endl;
}
```

함수 호출 연산자 중복
정의

실행결과

-100

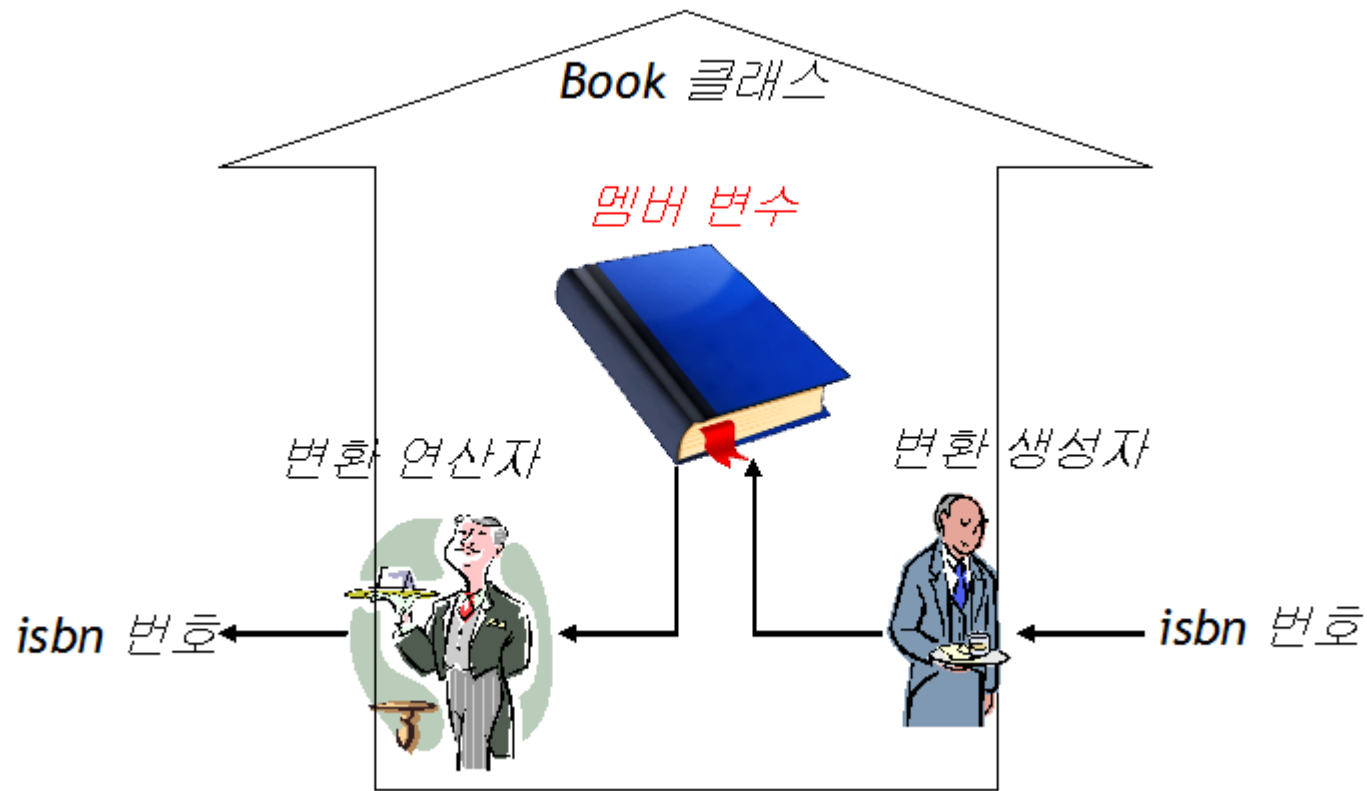
타입 변환

- 타입 변환
 - 클래스의 객체들도 하나의 타입에서 다른 타입으로 자동적인 변환이 가능하다.
 - 이것은 변환 생성자(conversion constructor)와 변환 연산자(conversion operator)에 의하여 가능하다.



타입 변환 (계속)

- 변환 생성자와 변환 연산자



타입 변환 (계속)

예제

```
#include <iostream>
#include <string>
using namespace std;

class Book
{
private:
    int isbn; // 책의ISBN
    string title; // 책의제목
public:
    Book() { // 생성자
        isbn = 0;
        title = "unknown";
    }
    Book(int isbn) {
        this->isbn = isbn;
        this->title = "unknown";
    }
    void display() {
        cout << isbn << ":" << title << endl;
    }
};
```

변환 생성자
(int->Book)

```
int main()
{
    Book b1 = 9782001; // int 타입을 Book 타입에 대입
    b1.display();
    b1 = 9783001; // 가능
    b1.display();
    return 0;
}
```

```
9782001:unknown
9783001:unknown
계속하려면 아무 키나 누르십시오 ...
```

컴파일러는 변환 생성자를 이용해서 정수 9782001을 Book 객체로 변환하는 것이다.

타입 변환 (계속)

- 변환 연산자 중복 정의

```
operator 데이터_타입()
{
....
}
```

```
class Book
{
    operator int() const
    {
        return isbn;
    }
}
```

변환 생성자
(int->Book)



타입 변환 (계속)

● 예제

```
#include <iostream>
#include <string>
using namespace std;
class Book
{
private:
```

```
    int isbn;
    string title;
```

```
public:
```

```
    Book(int isbn, string& title) {
        this->isbn = isbn;
        this->title = title;
    }
```

변환 생성자
(int->Book)

```
    Book(int isbn) {
        this->isbn = isbn;
        this->title = "unknown";
    }
```

변환 연산자
(Book->int)

```
    operator int() const
    {
        return isbn;
    }
```

```
void display() {
    cout << isbn << ":" << title << endl;
}
```

```
bool check(int isbn)
{
    cout << isbn << endl;
    return true;
}
```

```
int main()
{
    Book b1 = 9782001;           // 변환생성자실행!
    b1.display();
    int isbn = b1;               // 변환연산자실행!
    cout << isbn << endl;
    check(b1);                   // 변환연산자실행!

    return 0;
}
```

타입 변환 (계속)

- 연산자 중복시 주의할 점
 - 새로운 연산자를 만드는 것은 허용되지 않는다.
 - "::" 연산자, "." 연산자, ":" 연산자, "?:" 연산자는 중복이 불가능하다.
 - 내장된 int형이나 double형에 대한 연산자의 의미를 변경할 수는 없다.
 - 연산자들의 우선 순위나 결합 법칙은 변경되지 않는다.
 - 만약 "+" 연산자를 오버로딩하였다면 일관성을 위하여 "+=", "-=" 연산자도 오버로딩하는 것이 좋다.
 - 일반적으로 산술 연산자와 관계 연산자는 비멤버 함수로 정의한다. 반면 할당 연산자는 멤버 함수로 정의 한다.